Global and Stage Specific Concurrency Control for Agent Serving

Course Project Proposal

Lingxiao Yang Chenrui Gao

CS 395T: Hardware–Software Co-design for Cloud and Datacenters

Instructor: Jovan Stojkovic

Fall 2026 · Revised October 9, 2026

Motivation and research question

Tool-using agents repeatedly move between model inference and tool execution. An LLM output selects the next tool and its arguments; the tool result determines the next prompt, context length, and sometimes the remaining execution path. Scheduling one stage therefore changes the arrivals seen by the other. Separate concurrency limits provide additional control, but do not remove these dependencies.

We ask: **on a single CPU/GPU server, when is global request concurrency control sufficient, and when do separately adjustable stage limits improve end-to-end performance? Can an online policy follow changes in workload and resource availability?** CPU interference is the main diagnostic focus. GPU utilization, inference queues, correctness, and request latency are needed to evaluate its consequences; maximizing CPU utilization alone is not the objective.

The outcome will be an empirical map of operating conditions and an open-source serving testbed, together with a comparison of static and adaptive policies. A finding that global control is sufficient in a measured regime is useful evidence, not a failed attempt to demonstrate a predetermined speedup.

Revised hypotheses

H1: Resource balance changes the useful control policy. At fixed hardware, changing the available tool CPU budget or tool parallelism can change which stage constrains throughput. Stage-specific limits may help when they reduce contention without creating greater admission delay or inference starvation. These effects must be evaluated jointly, not subtracted as independently measured gains.

H2: There is no assumed universal tool-time threshold. Policy benefit depends on CPU capacity, tool type/threading, model service behavior, offered load, and trajectory structure. We will report the region where stage-specific control helps, ties, or hurts. A tool-time fraction is a descriptive measurement for a stated configuration, not an invariant workload property or a sufficient predictor.

H3: Online adaptation may help under changing demand. With the same observations and tuning budget, adaptive stage-specific control may outperform adaptive global control and tuned static policies during workload-mix or CPU-budget changes. We will measure recovery time, oscillation, and overhead as well as steady-state performance.

Scope and feasibility

The core study uses one server, one GPU, one model, and local tools. Remote CPU services, multi-node placement, and different GPU generations are outside the required evaluation. A second GPU platform is optional replication; reducing a CPU budget does not emulate a different GPU architecture.

Cheap tools remain a control case. For an isolated serial trajectory with unchanged service times, a small tool-time fraction bounds the direct benefit of speeding up that stage. It does not bound admission-control gains under queueing, batching, or interference. We will first measure live stage demand and static limit sweeps. If policies

are indistinguishable, report the tested boundary with uncertainty and use a lightweight adaptive implementation to quantify overhead rather than expanding into a complex scheduler.

Single Server Setup and Workloads

Execution stack

One Linux server hosts a load generator, a LangGraph ReAct-style data-analysis agent, one shared **Qwen3-8B/vLLM** instance, a CPU tool-executor service, and a trace collector. Model and tool calls use localhost HTTP. Plan for about 16 CPU cores, 64 GB RAM, and one GPU with 24-48 GB VRAM; actual access and completed implementation are not established by this proposal.

A shared executor uses a bounded process pool and FIFO tool queue. Inference submissions pass through a separate admission queue before vLLM. Resources for vLLM host threads, orchestration, and monitoring are reserved and recorded separately from the tool CPU budget. The same allocation is used for every policy comparison within a condition.

Pin model/tokenizer revisions, sampling configuration, context/token limits, prompts, software, and fixtures. Validate service health, tool-call parsing, gold outputs, trace IDs, and timeouts. Deploy with Docker Compose or a cluster-compatible environment inside the assigned allocation. If a smaller model is required, label it as a separate configuration: model substitution can change decisions and token counts, not just inference speed.

Verifiable task families

Family	Tool computation and validation	Role
Lightweight control	Indexed SQLite lookups and small aggregations; reference SQL answers.	Identify inference-dominated cases and controller overhead.
Data analysis	File parsing, joins, grouped aggregation, and numerical statistics in a Python worker; reference numeric results and hashes.	Candidate CPU/memory-intensive work; classify using measured profiles.

Use 10 calibration and 20 held-out tasks per family. Choose fixture sizes from declared task/data scales that fit the machine, retaining normal indexes and data types. Freeze fixtures before policy tuning and publish all sizes; do not select them to obtain a desired tool-time fraction or imitate an unavailable production distribution. Preserve errors, retries, and failed answers.

Controlled resource dimensions

Dimension	Core comparison	Interpretation
Tool CPU availability	For example 2, 4, and 8 dedicated cores on the same host	Changes effective CPU/GPU balance with model and GPU fixed.
Within-tool parallelism	One versus a small fixed thread count for tools that support it	Measures interaction between concurrent calls K and threads per call P.
Workload and arrivals	Light/heavy mixtures, steady arrivals, one burst and one mix shift	Tests policy robustness without changing the model.

Thread settings remain fixed during each policy comparison. A two-thread tool may not achieve twice the speed; measure scaling and CPU-seconds rather than infer demand from K alone. Start with serial tool dependen-

cies per request. Parallel fan-out within one agent is optional and must preserve dependency correctness. If affinity/cgroups are unavailable, report the actual isolation method and omit unsupported CPU-budget claims.

CPU and GPU capacities are profiled separately to explain the operating point, then validated jointly in live serving. GPU request latency is not the reciprocal of GPU request throughput because inference batches overlap. Record input/output tokens, action counts, bytes returned, and tool results so changes in execution path remain visible.

Policies and Continuous Monitoring

Admission controls

Control	Definition
N	Maximum active end-to-end agent requests, including requests waiting between stages.
K	Maximum executing tool calls; waiting calls occupy no worker.
M	Maximum model calls submitted to vLLM and not yet complete, including its internal waiting calls.

M is an external admission limit, not a GPU batch-size setting. A request holds a stage slot only for that call and releases it before entering another stage. Bound all queues; count admission waits and rejected arrivals. Decreasing a limit does not preempt work already running. Global policies tune N with nonbinding internal gates; stage policies hold a common outer safety cap and adjust K and M. Internal gates still obey fixed memory-safety limits.

Compare a tuned static global policy N, a tuned static stage policy (K, M), an adaptive global policy, and an adaptive stage policy. Default serving is a secondary reference. Add a K-only ablation at selected conditions to determine whether an inference-side gate adds value. This distinguishes the benefit of adaptation from the benefit of more control points.

Online controller

Both adaptive policies use the same lightweight feedback algorithm, initialization procedure, observation budget, and end-to-end objective: improve SLO goodput subject to fixed per-family latency and correctness requirements. The global policy can change N; the stage policy can change K or M. Offline calibration establishes safe ranges and initial values, while runtime measurements drive subsequent actions.

1. **Observe continuously.** Update queue depth, occupancy, arrivals, and completions on each event; sample process CPU and GPU/server metrics periodically. Maintain rolling tool-type service times, queue waits, and completion/goodput estimates.
2. **Choose a bounded probe.** When the window has enough observations, change one limit by one step. Pressure and starvation signals guide which direction to test. Low GPU utilization alone never triggers an increase.
3. **Evaluate the coupled effect.** Wait for enough affected work to complete before accepting another change. Retain a change only if the end-to-end objective improves without violating guards; otherwise revert. Use hysteresis, minimum dwell time, and hard bounds to limit oscillation.

This is one controller with separately adjustable stage limits, not two independent controllers optimizing local utilization. Fine-grained event monitoring does not require changing limits on every event. Decision windows scale with measured stage durations; sparse samples defer updates. Thresholds and window rules are fixed before held-out testing. Online control uses deadline-compliant completions as a goodput proxy; gold-answer correctness is checked separately by the benchmark and must not regress.

Profiling and attribution

OpenTelemetry links arrival, inference admission/submission/completion, tool enqueue/start/end, and response by request ID. py-spy and process/cgroup counters measure CPU hotspots/demand; perf/scheduler traces investigate contention where permitted. vLLM queue/token and GPU metrics provide inference context. Log policy actions and their effects.

Avoid double-counting overlapping spans or assigning aggregate CPU usage to one concurrent request. Separate tool computation, framework overhead, and waiting; validate accounting and measure tracing overhead. GPU idleness alone does not establish tool-induced starvation: require matched interventions and live confirmation. Report counter visibility limits.

Evaluation and Open Source Artifact

Experiments and success criteria

Start with a small static sweep over feasible N, K, and M values, rather than a full Cartesian search. Profile the CPU budgets and tool-thread settings on calibration tasks, then select representative inference-limited, tool-limited, and balanced conditions if these regimes actually occur. Use the same calibration allowance for all policies and freeze selections before held-out evaluation.

At selected operating points, compare all four policies under steady load and a workload-mix transition. Add one controlled CPU-budget reduction/recovery on the same host if allocation control is available. Keep the disturbance, absolute arrival rates, tasks, and total available resources identical across policies; controller actions never change the resource budget. Include an inference-heavy control even if it shows no benefit.

Measure **SLO goodput, successful throughput, p50/p95 end-to-end latency, and per-family correctness**. Set per-family SLOs before testing and count time from scheduled arrival, including every admission queue. Report rejected, timed-out, unfinished, and incorrect tasks. Also measure tool CPU-seconds, service/queue time, inference queueing, tokens, memory, and GPU activity. For transitions, report settling time, cumulative SLO misses, and limit oscillations.

Use three independent runs at selected conditions, randomized configuration order, and documented warmup/cache state. Target 500 or more completions per run for p95-led claims when compute permits, but sample count alone does not establish precision. Report uncertainty with task/run-aware resampling and increase samples only when needed to distinguish a material effect. If intervals remain wide, narrow the claim. Compare tracing and controller overhead separately.

The main figures are policy-benefit maps across CPU budget and tool parallelism, latency/goodput curves, and aligned timelines of resource changes, queues, and policy actions. Report effect sizes and uncertainty in place of one universal phi threshold. Hold workload/model fixed for hardware-allocation comparisons; report path/token changes in live policy comparisons.

Replay and simulation

Recorded model actions plus real tool execution support deterministic, paired policy comparisons; verify actions and tool-result hashes and fail visibly on divergence. Paced replay fixes model delays and therefore removes part of the very feedback under study. It is useful for debugging and isolating mechanisms, but cannot validate online CPU-GPU interaction, inference batching, or overall live gains. Core conclusions require the real local model. A calibrated trace-driven queue simulation is optional sensitivity analysis, not an additional required system.

Reusable release

Module	Released functionality
workloads/ and configs/	Fixtures, reference checks, resource/thread settings, and arrival/mix traces.
tracing/ and replay/	Stage schema, record/replay adapters, dependency checks, and CPU-only demo.

Module	Released functionality
policies/	Shared admission hooks, global/stage baselines, online controller, and event logs.
experiments/ and analysis/	Launch scripts, measured example outputs, confidence intervals, and figure generation.

Runs emit checked answers, trajectories, spans, request metrics, N/K/M events, and hardware/software manifests. Provide a clean-checkout guide and commands for replay, live serving, and plots; CI covers accounting, slot release, limit changes, and replay equivalence. Release original code under Apache-2.0 subject to dependency compatibility, retain upstream attribution, and publish sanitized fixtures/traces and model acquisition instructions. These are planned deliverables, not completed results.

Schedule and Research Context

Remaining semester plan

Planned milestones below use Austin local time; record actual progress and access in the midterm report.

Dates in 2026	Milestone and ownership
Oct 9-14	Confirm platform, trace the dependency loop, and run a limited static pilot. Chenrui: tasks/checks; Lingxiao: resources/profiling.
Oct 15/20; report Oct 22, 9 a.m.	Midterm presentation slot and report: revised scope, available evidence, unresolved access/measurement issues. Both.
Oct 23-Nov 8	Implement shared admission hooks and both online policies; calibrate ranges/windows. Lingxiao: policies; Chenrui: load/replay.
Nov 9-22	Held-out stationary and transition experiments, ablations, uncertainty analysis. Both.
Nov 23-Dec 8	Clean reproduction, artifact release, and presentation preparation. Confirm final presentation slot with course staff. Both.
Dec 9-15	Final report and limitations; report due Dec 15, 9 a.m. Both.

Plan 8-10 hours/person/week. Reduce operating points if compute is limited; retain live validation and report policy overhead or lack of benefit.

Relation to prior work

The course readings [1-3] motivate profiling, application-level headroom, and workload-sensitive management. GrandSLAm [4] motivates stage-level latency management; μTune [5] motivates load adaptation. Shenango [6] reallocates CPU cores, while Splitwise [7] separates inference phases; our controller does neither. Raj et al. [8] already study agent CPU-aware batching/scheduling, and AgentSysBench [9] studies heterogeneous components and deployment sensitivity. We will reuse compatible artifacts. Our scoped contribution is a reproducible comparison of global versus stage-specific online control across measured resource conditions, without claiming that prior work overlooks this dependence.

References

[1] Kanev et al. **Profiling a Warehouse-Scale Computer.** ISCA 2015. Publication.

[2] Sriraman and Dhanotia. **Accelerometer: Understanding Acceleration Opportunities for Data Center Overheads at Hyperscale.** ASPLOS 2020. Artifact.

[3] Shahrad et al. **Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider.** USENIX ATC 2020. Paper.

[4] Kannan et al. **GrandSLAm: Guaranteeing SLAs for Jobs in Microservices Execution Frameworks.** EuroSys 2019. Paper.

[5] Sriraman and Wenisch. **μTune: Auto-Tuned Threading for OLDI Microservices.** OSDI 2018. Paper.

[6] Ousterhout et al. **Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads.** NSDI 2019. Paper.

[7] Patel et al. **Splitwise: Efficient Generative LLM Inference Using Phase Splitting.** ISCA 2024. Paper.

[8] Raj et al. **Towards Understanding, Analyzing, and Optimizing Agentic AI Execution: A CPU-Centric Perspective.** arXiv:2511.00739v3, 2026 revision. Paper.

[9] Chang et al. **From LLM Inference to Agentic Workloads: Characterization and Implications for Serving Systems.** AgentSysBench, arXiv:2608.15127, 2026. Paper.

AI assistance disclosure: ChatGPT Codex assisted with research, design, writing, and revision; the prior draft disclosed Claude critique/editing. No results are asserted. Authors must review claims and disclose subsequent AI use.